**Sharif University of Technology**

Department of Electrical Engineering

Bachelor's Degree Project Report

# Recognition and Continuous Estimation of Three Wrist Motions Using Surface Electromyography Signals

Student: Reza Sanaeinejad

Supervisor: Dr. Mehran Jahed

Mordad 1405 (July–August 2026)

Complete English Report with Supplementary Technical Analysis

# Abstract

This project presents the design and implementation of a system for recognizing and continuously estimating three wrist motions, namely inward and outward rotation (Roll), upward and downward flexion (Pitch), and leftward and rightward deviation (Yaw), using surface electromyography (sEMG) signals. The data were obtained from the DB2 subset of the standard NinaPro database, which records muscle signals at a sampling frequency of 2,000 Hz together with corresponding kinematic data acquired using a Cyberglove II instrumented glove.

Model development proceeded through four consecutive phases. In the first phase, a hybrid architecture combining one-dimensional convolutional networks and long short-term memory (LSTM) networks was designed for a single subject under an intra-subject protocol and trained using the Huber loss function. In the second phase, data from seven different subjects were pooled, and the model was evaluated for inter-subject generalization, which resulted in a substantial decline in accuracy. In the third phase, dilated convolutions were introduced, and the Leave-One-Subject-Out and Global Repetition Split validation protocols were examined, providing experimental evidence of covariate shift in sEMG signals across individuals. In the fourth and final phase, the study returned to a subject-specific approach. The final model was designed and evaluated using a Gaussian low-pass filter to smooth discrete labels, together with a gain calibration technique to compensate for attenuation of the output amplitude.

Quantitative results on the test data indicate Pearson correlation coefficients of 0.8707, 0.9634, and 0.8949 for the Roll, Pitch, and Yaw axes, respectively. These results demonstrate a high degree of agreement between the model output and the actual hand motion.

Keywords: surface electromyography, continuous kinematic estimation, convolutional recurrent neural network, NinaPro database, covariate shift, gain calibration

# Contents

Abstract....2
Chapter 1 Introduction....7
1.1 Motivation and Significance....7
1.2 Problem Definition....7
1.3 Related Research....7
1.4 Project Contributions....8
1.5 Report Organization....8
Chapter 2 Theoretical Foundations and the Database....9
2.1 Surface Electromyography Signals....9
2.2 Wrist Kinematics....9
2.3 Introduction to the NinaPro DB2 Database....9
2.3.1 Participants....10
2.3.2 Data Acquisition Equipment....10
2.3.3 Movement Protocol and Experimental Procedure....10
2.3.4 Movement Categories....11
2.3.5 Alignment of the Data with the Research Objectives....11
Chapter 3 Research Methodology and Model Development....12
3.1 Phase 1 Development of the Baseline Intra Subject Model....12
3.1.1 Development of Data Preprocessing....12
3.1.2 Development of the Neural Network Architecture and Loss Functions....12
3.1.3 Key Decisions in This Phase....13
3.1.4 Achievements and Limitations of Phase 1....13
3.2 Phase 2 Evaluation of the Generalized Inter Subject Model....13
3.2.1 Pooling and Integration of Multiple Datasets....14
3.2.2 Architectural Development for Complex Data....14
3.2.3 Learning Rate Control Strategy....14
3.2.4 Achievements and Limitations of Phase 2....14
3.3 Phase 3 Improvements to the Spatiotemporal Architecture and Validation Strategies....15
3.3.1 Data Partitioning Strategies....15
3.3.2 Architectural Improvements Using Dilated Convolutions....15
3.3.3 Advanced Kinematic Postprocessing....16
3.3.4 Achievements and the Fundamental Challenge of Phase 3....16
3.4 Phase 4 Transition to Continuous Kinematics and Final Calibration....18
3.4.1 Smoothing the Target Labels....18
3.4.2 Initial Evaluation and Amplitude Attenuation....19
3.4.3 Gain Calibration and Output Clipping....19
3.4.4 Final Model Architecture....20
Chapter 4 Results and Evaluation....21
4.1 Quantitative Results of the Final Model....21
4.2 Visual Analysis of the Results....21
4.3 Discussion....22
Chapter 5 Conclusions....23
5.1 Overview of the Findings....23

5.2 Limitations....23
5.3 Recommendations for Future Research....24
Chapter 6 Verification of the Supplied Implementation....25
6.1 Evidence and Reproducibility Scope....25
6.2 Target Construction and the Meaning of the Outputs....26
6.3 Preprocessing and Windowing Audit....26
6.4 Verified Final Architecture and Parameter Counts....27
6.5 Training and Evaluation Provenance....28
6.6 Comparison with the Supplied Literature....29
Chapter 7 Extended Evaluation of the Saved Predictions....31
7.1 Data Integrity and Evaluation Definitions....31
7.2 Relationship to the Originally Reported Results....32
7.3 Residual Behavior and Large Errors....33
7.4 Prediction and Target Agreement....34
7.5 Activity Conditioned Evaluation and the Zero Baseline....34
Chapter 8 Reproducible Software and Further Research Design....36
8.1 Organization of the Research Implementation....36
8.2 Verification of Numerical Equivalence....36
8.3 Independent Training Validation and Test Partitions....37
8.4 Measured Kinematic Supervision and Physical Calibration....37
8.5 Ablation Experiments and Baseline Comparisons....38
8.6 Causal Streaming and Deployment Evaluation....38
8.7 Conditions for Further Scientific Conclusions....39
References....40

# List of Figures

Figure 2.1 Three wrist degrees of freedom: rotation (Roll), flexion (Pitch), and deviation (Yaw) ..... 9
Figure 3.1 Baseline model development pipeline in Phase 1 under the intra-subject protocol ..... 12
Figure 3.2 Performance of the Phase 1 baseline model on the test data of a single subject ..... 13
Figure 3.3 Output of the generalized Phase 2 model on the pooled data of seven subjects ..... 15
Figure 3.4 Dilated convolutions and expansion of the network's receptive field ..... 16
Figure 3.5 Output of the Phase 3 model under the Leave-One-Subject-Out protocol for a completely new subject ..... 17
Figure 3.6 Output of the Phase 3 model under the Global Repetition Split protocol ..... 18
Figure 3.7 Smoothing discrete stepwise labels into continuous kinematic trajectories using a Gaussian low-pass filter ..... 19
Figure 3.8 Effect of gain calibration on compensating for attenuation of the model's output amplitude ..... 19
Figure 3.9 Detailed architecture of the final hybrid CNN-LSTM neural network ..... 20
Figure 4.1 Agreement between actual and predicted angles of the final model on the test data ..... 22
Figure 5.1 Overview of the project's development through four phases ..... 23
Figure 7.1 Residual trajectories and error distributions for the saved calibrated predictions ..... 33
Figure 7.2 Scatter comparison between smoothed targets and saved calibrated predictions ..... 34
Figure 7.3 Zero-baseline comparison and activity-conditioned RMSE ..... 35

# List of Tables

Table 2.1 Participant characteristics and data acquisition equipment in the NinaPro DB2 database ..... 10
Table 2.2 Movement categories in the NinaPro DB2 database ..... 11
Table 3.1 Comparison of network architecture complexity in the two versions of Phase 2 ..... 14
Table 4.1 Quantitative evaluation results of the final model on the test data ..... 21
Table 6.1 Evidence available for verification ..... 25
Table 6.2 Movement code mapping used by the implementation ..... 26
Table 6.3 Layer dimensions and parameter counts in the final model ..... 27
Table 6.4 Methodological relevance of the supplied references ..... 30
Table 7.1 Extended metrics for all saved calibrated predictions ..... 31
Table 7.2 Original pre-calibration results and saved calibrated results ..... 32
Table 7.3 Bias and absolute-error tails ..... 33
Table 7.4 Activity counts and conditional errors ..... 34
Table 7.5 Constant zero baseline and prediction saturation ..... 35
Table 8.1 Responsibilities of the organized Python modules ..... 36
Table 8.2 Proposed experiments and the questions they address ..... 38

# Chapter 1 Introduction

## 1.1 Motivation and Significance

Communication between the central nervous system and skeletal muscles occurs through electrical signals. Recording these signals at the skin surface produces surface electromyography, or sEMG. Because sEMG is directly related to muscular activity, it is one of the most important sources of information for the design of human–machine interfaces, intelligent prostheses, and rehabilitation systems.

The high noise level, stochastic nature, and strong dependence of sEMG signals on anatomical and physiological differences between individuals create significant challenges for signal processing and the extraction of precise motion information. Many existing systems recognize motion by performing discrete classification among a limited set of predefined movements. However, natural human hand motion is continuous, and joint angles change smoothly and gradually. This discrepancy between the discrete outputs of conventional models and the continuous nature of actual motion creates a need for models that can continuously estimate joint motion angles in real time.

## 1.2 Problem Definition

The principal objective of this project is to design and train a deep learning model that receives sEMG signals recorded from the forearm muscles and continuously and simultaneously estimates three wrist degrees of freedom: inward and outward rotation (Roll), upward and downward flexion (Pitch), and leftward and rightward deviation (Yaw). Unlike conventional approaches that classify discrete movements, this study seeks to provide continuous angular values within the normalized range from −1 to +1 for all three motion axes, allowing the model output to be used in systems for real-time control of hand prostheses or human–machine interfaces.

## 1.3 Related Research

Numerous studies have examined the extraction of motion information from sEMG signals. By introducing and comprehensively characterizing the NinaPro database, Atzori et al. established a standard reference for evaluating machine learning methods based on the simultaneous recording of muscle signals and hand kinematic data [1]. Subsequently, Gijsberts et al. introduced the movement error rate metric and extensively compared the performance of several machine learning methods on the same database [2]. Bao et al. proposed a hybrid CNN-LSTM architecture for estimating wrist kinematics from sEMG signals. In their framework, a convolutional network extracts deep features from the signal spectrum, after which an LSTM network processes these features for sequential regression [3]. In a related study, Quivira et al. used recurrent neural networks to translate sEMG signals into continuous hand poses [4].

In research closely related to this project, Du et al. proposed a hybrid TCN-LSTM model based on dilated convolutions for continuously estimating wrist joint angles from sEMG signals [5]. Ameri et al. also demonstrated improvements in simultaneous and proportional EMG control by proposing a regression convolutional neural network [6]. Finally, Atzori et al. applied deep convolutional neural networks to the NinaPro database and evaluated their ability to classify hand movements for intelligent prostheses [7]. Despite this substantial body of research, generalization across individuals remains an open challenge in this field. This issue was also investigated experimentally during the intermediate phases of the present project.

## 1.4 Project Contributions

The main contributions of this project are as follows:

- A transition from discrete movement classification to continuous estimation of three wrist angles through label smoothing using a Gaussian low-pass filter.
- Experimental investigation and practical demonstration of covariate shift in sEMG signals across individuals by comparing the Leave-One-Subject-Out and Global Repetition Split validation protocols.
- Design of a hybrid CNN-LSTM architecture with dilated convolutions to expand the network's temporal receptive field without increasing the computational burden.
- Introduction of a gain calibration technique to compensate for the conservative behavior induced by the MSE loss function and to bring the prediction amplitude closer to the actual movement values.

## 1.5 Report Organization

The remainder of this report is organized into four additional chapters. Chapter 2 presents the theoretical foundations of sEMG signals and wrist kinematics, together with a comprehensive introduction to the NinaPro DB2 database. Chapter 3 describes the complete development of the model through four research phases. Chapter 4 presents and analyzes the quantitative and qualitative results of the final model. Finally, Chapter 5 provides the conclusions, discusses the limitations, and proposes directions for future research.

# Chapter 2 Theoretical Foundations and the Database

## 2.1 Surface Electromyography Signals

Surface electromyography records the electrical activity of muscles at the skin surface. Because these signals are nonstationary and contain motion and electrical noise, careful preprocessing is required before feature extraction. In this project, a Butterworth low-pass filter with a cutoff frequency of 5 Hz was combined with signal rectification using the absolute value to extract the signal's energy envelope.

## 2.2 Wrist Kinematics

The human wrist has three principal degrees of freedom. The Roll axis represents inward and outward rotation of the forearm. The Pitch axis represents upward and downward wrist flexion. The Yaw axis represents leftward and rightward wrist deviation. The objective of this project is to estimate all three angles continuously and simultaneously from sEMG signals.

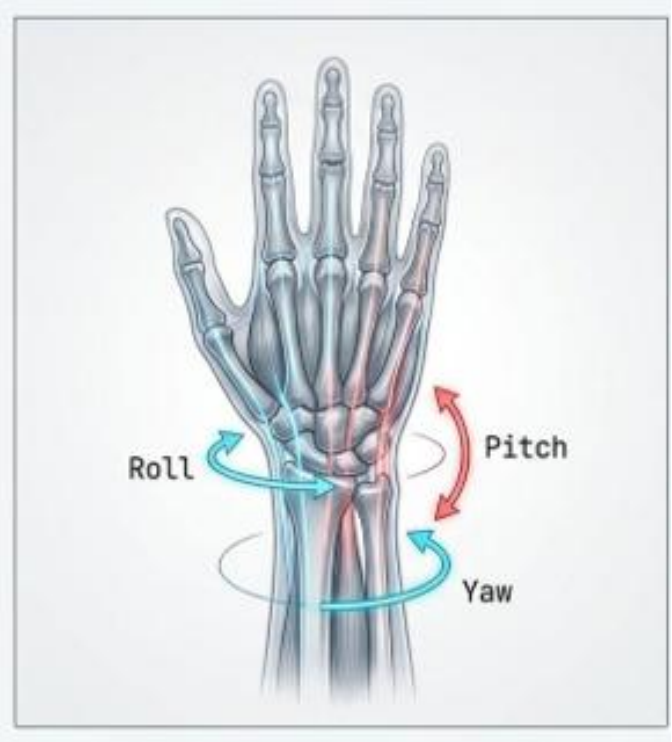



Figure 2.1 Three wrist degrees of freedom: rotation (Roll), flexion (Pitch), and deviation (Yaw)

## 2.3 Introduction to the NinaPro DB2 Database

The standard NinaPro DB2 database was used to train and evaluate the deep learning models in this study. The NinaPro project was developed to provide a comprehensive reference collection of surface electromyography signals and hand kinematic data for advancing research on intelligent prostheses and human–machine interfaces. The DB2 subset specifically focuses on able-bodied individuals and is one of the richest available datasets in terms of movement diversity and equipment precision.

### 2.3.1 Participants

The DB2 database was collected from 40 able-bodied participants, including 28 men and 12 women, with a mean age of 29.9 years. All participants were right-handed and had no history of neurological or muscular disease. For model evaluation in this study, a subset of seven subjects, numbered 1, 7, 8, 11, 22, 25, and 35, was selected and processed because their recordings exhibited the highest signal quality and the lowest levels of motion noise.

### 2.3.2 Data Acquisition Equipment

The database acquisition system comprises two main components for measuring muscular activity and physical movement. The Delsys Trigno EMG acquisition system recorded muscle electrical activity using 12 wireless electrodes at a sampling frequency of 2,000 Hz. Eight electrodes were placed at equal intervals in a ring around the upper forearm, two electrodes were positioned over the principal flexor and extensor muscles, and two additional electrodes were placed over the biceps and triceps to record compound hand contractions. Simultaneously, a Cyberglove II kinematic acquisition system equipped with 22 sensors continuously recorded precise finger and wrist joint angles, enabling their use as regression labels when required.

Table 2.1 Participant characteristics and data acquisition equipment in the NinaPro DB2 database

| Parameter | Value |
|---|---|
| Total number of database participants | 40 participants (28 men and 12 women) |
| Mean participant age | 29.9 years |
| Subset selected for this study | 7 subjects (1, 7, 8, 11, 22, 25, and 35) |
| EMG acquisition system | Delsys Trigno with 12 wireless electrodes |
| EMG sampling frequency | 2,000 Hz |
| Kinematic acquisition system | Cyberglove II with 22 sensors |
| Number of repetitions per movement | 6 repetitions (5 seconds of execution and 3 seconds of rest) |

### 2.3.3 Movement Protocol and Experimental Procedure

The experimental protocol was designed to minimize muscular fatigue while providing sufficient data for machine learning. Participants were instructed to sit in front of a monitor and imitate movements demonstrated in a video animation. Each specified movement was repeated exactly six times. During each repetition, the participant performed and maintained the movement for five seconds and then returned to a resting state for three seconds to prevent muscular fatigue.

### 2.3.4 Movement Categories

The DB2 dataset contains 50 different hand and wrist movements divided into three principal categories. Category A contains 17 basic, isolated finger and wrist movements. Category B contains 23 more complex movements, grasping patterns, and activities of daily living. Category C contains nine force application and object grasping patterns.

Table 2.2 Movement categories in the NinaPro DB2 database

| Movement category | Number of movements | Description |
|---|---|---|
| Exercise A | 17 movements | Basic, isolated finger and wrist movements |
| Exercise B | 23 movements | Grasping patterns and activities of daily living |
| Exercise C | 9 movements | Force application and object grasping patterns |
| Total | 50 movements | All movements in the NinaPro DB2 dataset |

### 2.3.5 Alignment of the Data with the Research Objectives

Because the objective of this study was to continuously estimate three wrist motions, only movements associated with the Roll, Pitch, and Yaw degrees of freedom were filtered and extracted from the 50 available movements. In addition, postprocessing techniques and biomechanical filters, including Gaussian filtering of the labels, were used to convert the database's discrete, visually defined labels into continuous and natural motion trajectories, enabling regression models to support real-time control.

# Chapter 3 Research Methodology and Model Development

The model was developed progressively through four consecutive research phases. In each phase, the results and limitations observed in the preceding phase informed a scientific decision to modify the architecture, the data, or the validation strategy. Each phase is described in detail in the following sections.

## 3.1 Phase 1 Development of the Baseline Intra Subject Model

The objective of this phase was to construct a baseline architecture that was robust to sEMG signal noise and prevented data leakage. An intra-subject training approach was adopted: the model was trained on repetitions 1, 3, 4, and 6 from subject 1 and evaluated on repetitions 2 and 5 from the same subject.

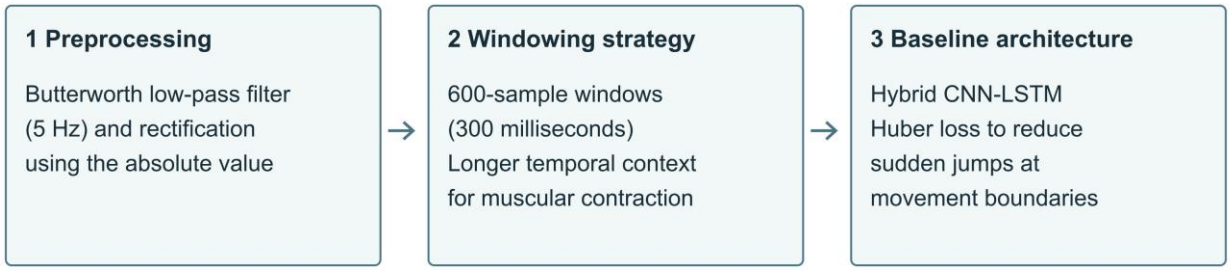



Figure 3.1 Baseline model development pipeline in Phase 1 under the intra-subject protocol

### 3.1.1 Development of Data Preprocessing

In the first version of this phase, the raw signal was used without amplitude filtering, and time windows were labeled using the mean label value within each window. This produced fractional labels at movement boundaries and confused the network when identifying the exact onset of motion. Two fundamental changes were introduced in the second version. First, the absolute signal value was used for rectification so that the signal's energy amplitude would be more interpretable to the network. Second, StandardScaler normalization was fitted exclusively to the training data and then applied to the test data to prevent data leakage. In addition, each window's label was changed to the label of its final sample so that prediction respected the principle of causality. In the fourth version, the time window was increased from 400 to 600 samples, corresponding to 300 milliseconds at a sampling frequency of 2,000 Hz. This allowed the network to observe a longer temporal context of muscular contraction before making a prediction.

### 3.1.2 Development of the Neural Network Architecture and Loss Functions

The model architecture began as a simple combination of one-dimensional convolution and a unidirectional LSTM with a mean squared error loss function. Because the labels were stepwise in this phase, the network exhibited substantial oscillations at movement boundaries. Consequently, the second version replaced the loss function with Huber loss, which combines linear and quadratic error terms and is more robust to abrupt changes. In the third version, conventional Dropout layers were replaced with SpatialDropout1D. Rather than disabling randomly selected neurons, this layer disables an entire feature map and is more effective for correlated EMG channel signals. The unidirectional LSTM was also upgraded to a bidirectional LSTM so that the network could analyze signal patterns in both the forward and backward directions within a window.

In the fourth version, once the architecture had stabilized, the number of training epochs was increased from 25 to 60. The model then reduced the validation error to an acceptable level without severe overfitting.

### 3.1.3 Key Decisions in This Phase

Two fundamental decisions brought the results closer to the standards reported in established research publications. First, the intra-subject protocol replaced the pooling of data from multiple individuals to avoid the substantial loss of accuracy caused by covariate shift. Second, from the middle of this phase onward, a moving-average filter with a specified window length was applied to the predicted outputs. This considerably smoothed the output curve and made it suitable for more precise motion recognition.

### 3.1.4 Achievements and Limitations of Phase 1

This phase established a robust baseline architecture and addressed data leakage, resulting in stable model outputs for a single individual. However, the fundamental limitation was that the model remained restricted to data from one particular user. This limitation motivated Phase 2, which investigated whether the architecture could be extended by pooling data from multiple users to obtain a more general model.

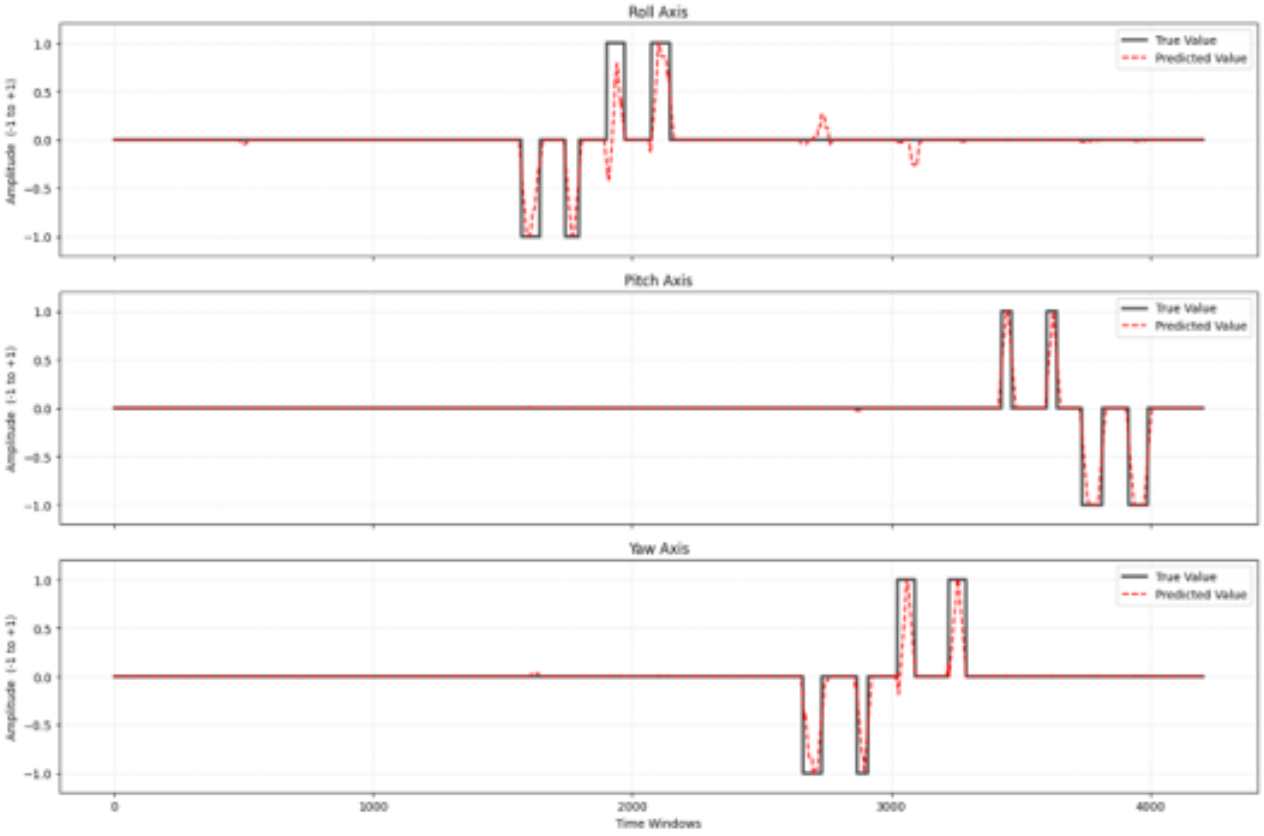



Figure 3.2 Performance of the Phase 1 baseline model on the test data of a single subject

## 3.2 Phase 2 Evaluation of the Generalized Inter Subject Model

In this phase, the research approach shifted from a single individual toward the development of a general model. The main objective was to pool data from multiple users and train a network capable of correctly recognizing movement patterns despite anatomical and physiological differences between individuals.

### 3.2.1 Pooling and Integration of Multiple Datasets

Data from seven different subjects were processed to construct a comprehensive model. At this stage, each subject's training and test data were separately extracted, filtered, and rectified, and were subsequently combined using array concatenation. Data were segmented into windows of 600 samples with a step of 50 samples. This produced a global training matrix containing more than 107,000 time windows and a test matrix containing approximately 52,000 windows.

### 3.2.2 Architectural Development for Complex Data

The substantial increase in diversity within the pooled data required a major architectural upgrade. The first version used a simpler structure consisting of one-dimensional convolution layers, max pooling, conventional Dropout layers with a rate of 0.2, and a unidirectional LSTM. This architecture contained 134,291 trainable parameters and had difficulty identifying stable patterns across the diverse signals of different individuals. In the second version, the conventional Dropout layers were replaced with SpatialDropout1D. The most important change was the upgrade of the recurrent layer to a bidirectional LSTM, which improved the model's ability to understand the signal's temporal sequence in both directions. These modifications increased the number of network parameters to more than 238,000.

Table 3.1 Comparison of network architecture complexity in the two versions of Phase 2

| Version | Recurrent layer | Dropout type | Trainable parameters |
|---|---|---|---|
| First version (V1) | Unidirectional LSTM | Conventional Dropout (rate 0.2) | 134,291 |
| Second version (V2) | Bidirectional LSTM | SpatialDropout1D | More than 238,000 |

### 3.2.3 Learning Rate Control Strategy

As the network grew and the loss landscape became more complex, EarlyStopping with a patience of 15 epochs was used to prevent overfitting. ReduceLROnPlateau was also used to reduce the initial learning rate of 0.001 by a factor of 0.5 when the validation error stopped decreasing. This produced smoother convergence during the final training epochs.

### 3.2.4 Achievements and Limitations of Phase 2

This phase produced a deeper network capable of processing the large volume of data from seven users without interruption or severe overfitting. Nevertheless, the principal weakness at this stage was the network's inability to fully capture long-term signal dependencies amid this high level of diversity, which reduced prediction accuracy. This challenge motivated Phase 3, in which dilated convolutions were introduced to expand the network's receptive field and inter-subject validation strategies were evaluated in greater detail.

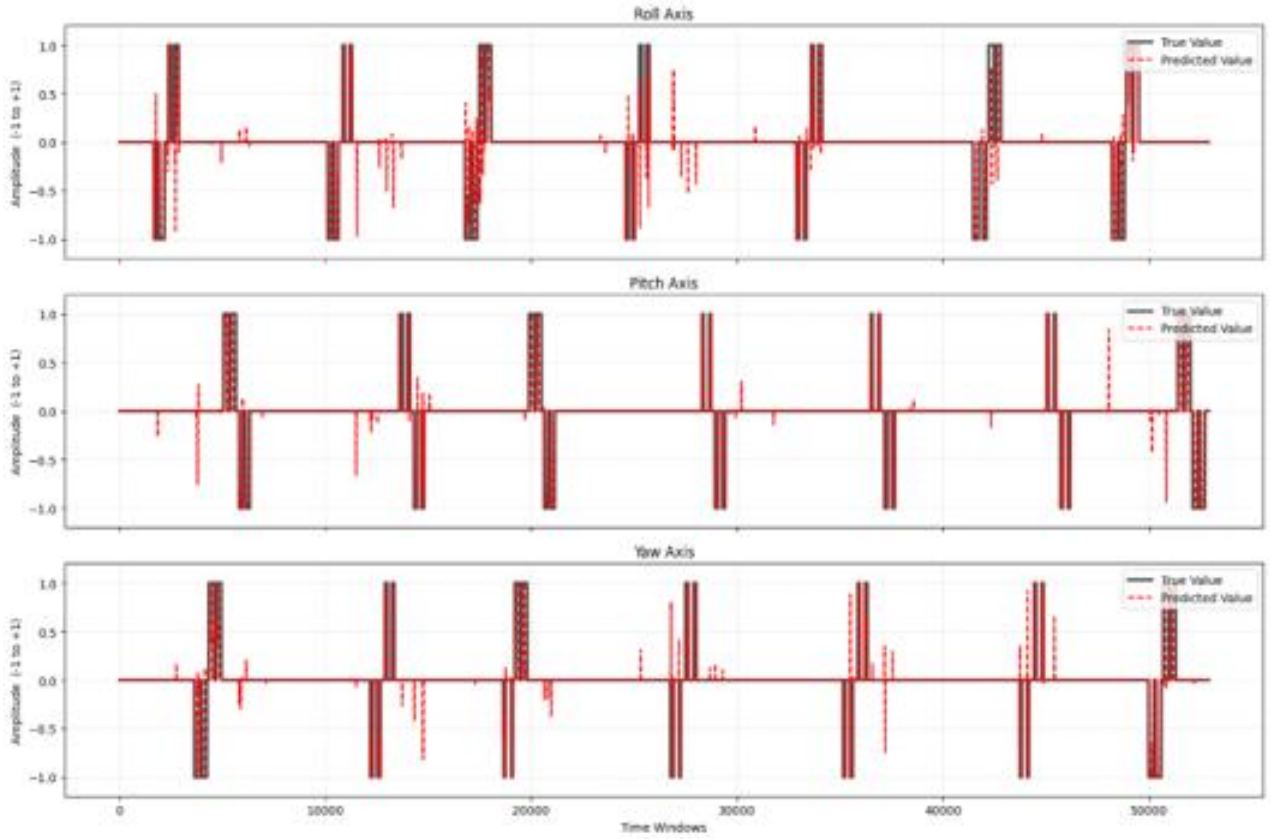



Figure 3.3 Output of the generalized Phase 2 model on the pooled data of seven subjects

## 3.3 Phase 3 Improvements to the Spatiotemporal Architecture and Validation Strategies

Phase 3 focused on expanding the network's receptive field and examining different cross-validation protocols to address inter-subject differences.

### 3.3.1 Data Partitioning Strategies

Under the Leave-One-Subject-Out protocol, the network was trained on six subjects, while one subject was held out entirely for testing. Because the model encountered completely new anatomy, fine-tuning and transfer learning were implemented, with only the final dense layers trained using a limited amount of data from the new subject. Under the Global Repetition Split protocol, repetitions 1, 3, 4, and 6 from all seven subjects were used for training, and repetitions 2 and 5 from all subjects were used for testing. This allowed the model to become familiar with the signal diversity of all individuals simultaneously.

### 3.3.2 Architectural Improvements Using Dilated Convolutions

The dilation rate was introduced into the architecture to enable the network to capture a longer period of muscular contraction. Three convolution layers with increasing dilation rates of 1, 2, and 4 were used in this phase, allowing the network to extract high-frequency features and the overall movement trend simultaneously. GlobalAveragePooling1D was also used in the aggregation stage to increase the model's robustness to temporal shifts in the signal.

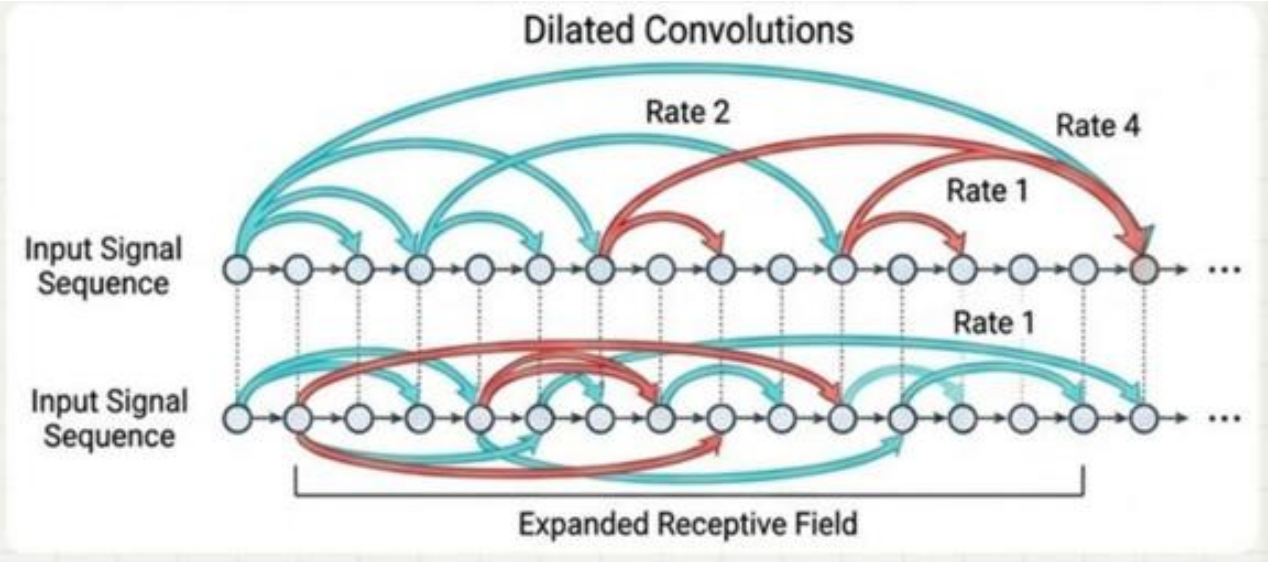



Figure 3.4 Dilated convolutions and expansion of the network's receptive field

### 3.3.3 Advanced Kinematic Postprocessing

A Savitzky–Golay filter replaced the simple moving-average filter. This filter removed high-frequency noise from the neural network's prediction curve without eliminating the peaks required for movement recognition.

### 3.3.4 Achievements and the Fundamental Challenge of Phase 3

In Phase 3, the network architecture reached its highest level of maturity in feature extraction. However, analysis of the results revealed a fundamental scientific challenge: regardless of the extent of training on pooled samples from multiple subjects, recognition performance remained very low for a completely new individual. Covariate shift in sEMG signals across individuals was so substantial that constructing a fully general model appeared practically impossible, even when transfer learning was applied to the final layers. This generalization impasse redirected the overall project strategy toward a subject-specific approach in Phase 4, this time with the objective of fully continuous recognition rather than discrete classification.

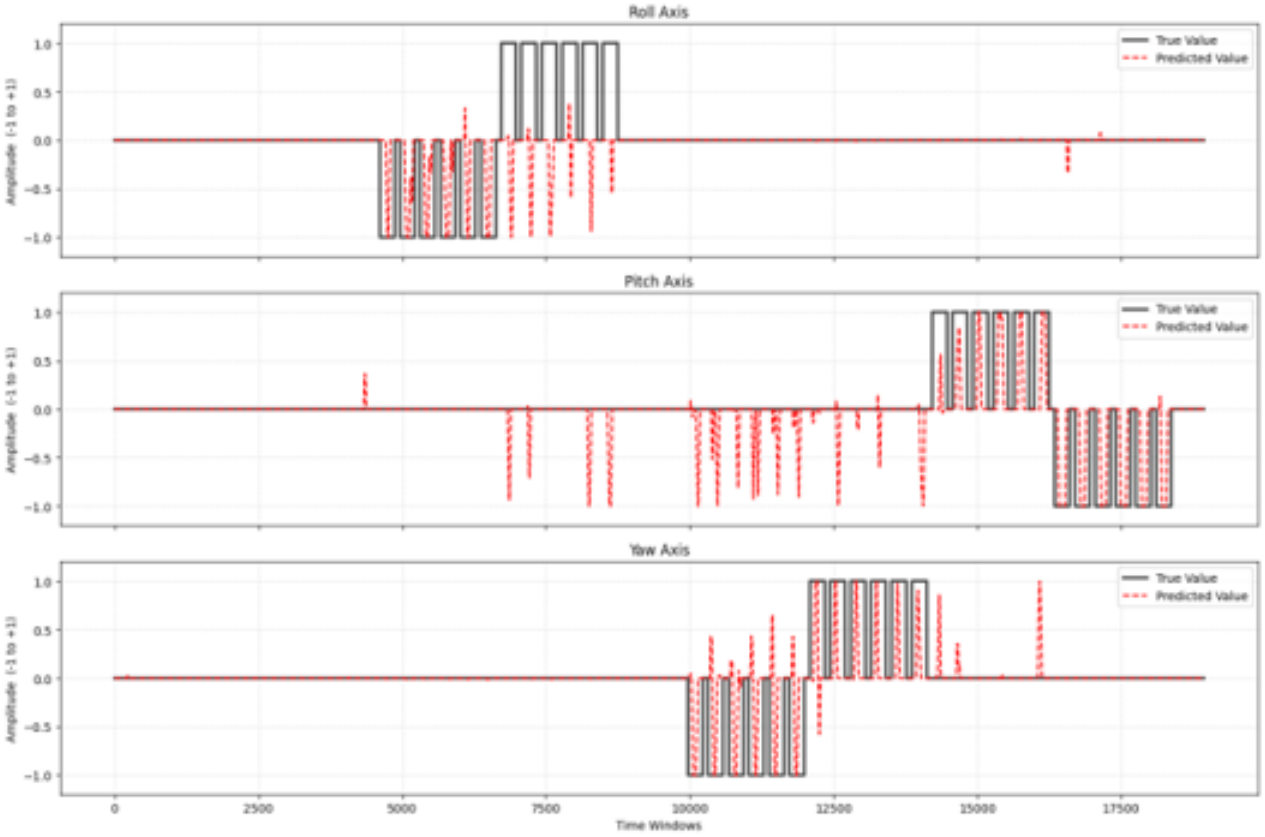



Figure 3.5 Output of the Phase 3 model under the Leave-One-Subject-Out protocol for a completely new subject

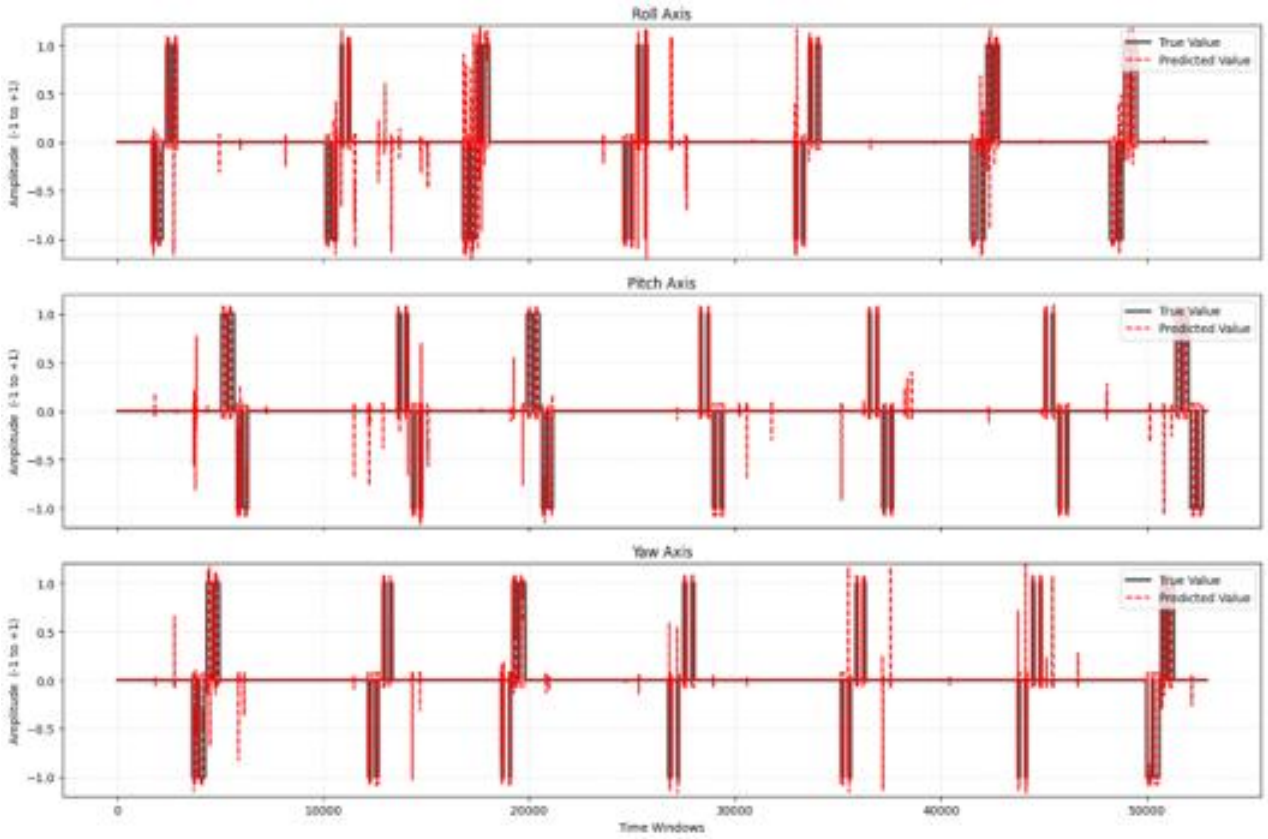



Figure 3.6 Output of the Phase 3 model under the Global Repetition Split protocol

## 3.4 Phase 4 Transition to Continuous Kinematics and Final Calibration

The shift to intra-subject training to overcome users' physiological differences transformed the project from a simple discrete classification problem into a continuous estimation system. Because a human hand cannot change its angle instantaneously in a stepwise manner, fitting the sharp edges of the original labels produced oscillations. Two innovations introduced in this phase addressed this problem.

### 3.4.1 Smoothing the Target Labels

At this stage, the previous label-generation function was replaced with a new function that applied a Gaussian low-pass filter with a sigma of 400 to the stepwise labels. This innovation caused the network's targets to simulate natural muscular behavior, including gradual acceleration, reaching a peak, and smooth deceleration to a stop. Rather than memorizing discontinuities, the neural network identified a coherent, continuous motion trajectory for all three angles.

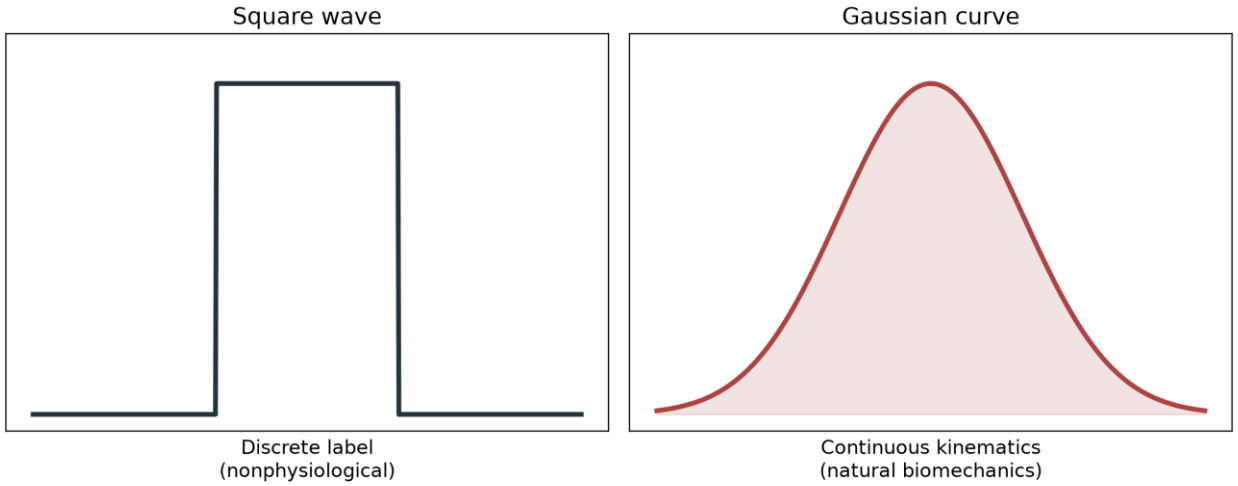



Figure 3.7 Smoothing discrete stepwise labels into continuous kinematic trajectories using a Gaussian low-pass filter

### 3.4.2 Initial Evaluation and Amplitude Attenuation

Applying the model to the smoothed targets produced substantial improvements in temporal alignment, and the Pearson correlation coefficients for the motion axes reached high values. However, the use of the MSE loss function and weight penalties induced conservative behavior: at the peaks of movement, the model stopped at lower amplitudes instead of reaching the absolute target values.

### 3.4.3 Gain Calibration and Output Clipping

Gain calibration was used to address the failure to reach movement peaks without disrupting temporal correlation. To match the prediction amplitude to the actual movement amplitude, a gain factor of 1.35 was applied to the model output, and the resulting values were clipped to the permitted range from −1 to +1. When clipping limited the prediction, this technique produced a flat plateau at the peaks of the curve, ensuring stable motion recognition at the extreme angles.

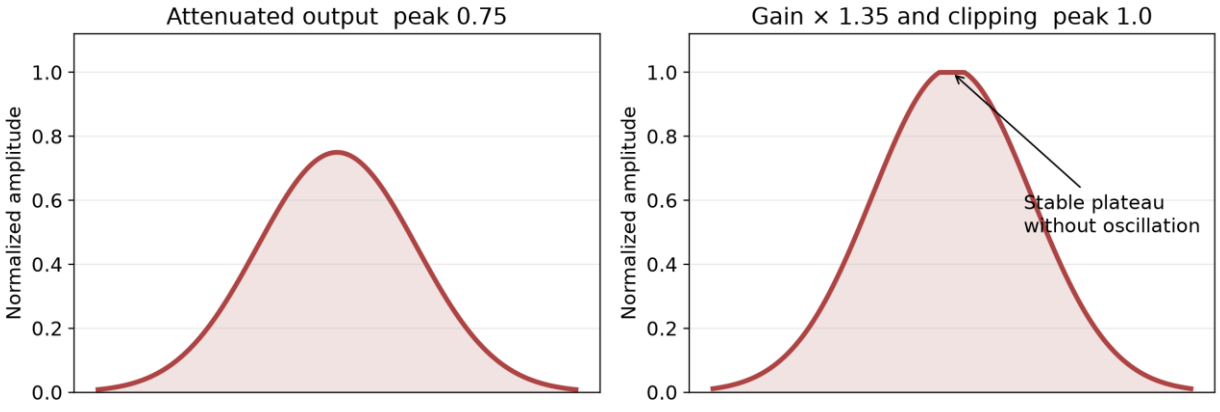



Figure 3.8 Effect of gain calibration on compensating for attenuation of the model's output amplitude

### 3.4.4 Final Model Architecture

The final model combines CNN and LSTM layers because a standalone CNN does not capture the movement sequence, whereas a standalone LSTM is computationally demanding and performs poorly when extracting features from highly fluctuating raw signals. The model layers are arranged as follows:

- Addition of random Gaussian noise at the input to improve robustness to sensor noise under real-world conditions.
- Two Conv1D layers with L2 regularization to extract high-frequency spatial features.
- BatchNormalization and MaxPooling layers to accelerate convergence and reduce dimensionality.
- A SpatialDropout1D layer with a rate of 0.2 to prevent overfitting in spatial feature maps.
- An LSTM layer with 64 units to extract complex, long-term temporal dependencies.
- A GlobalAveragePooling1D layer for robust feature aggregation.
- An output Dense layer mapping 64 features to 3 outputs for simultaneous linear regression of the three motion axes.

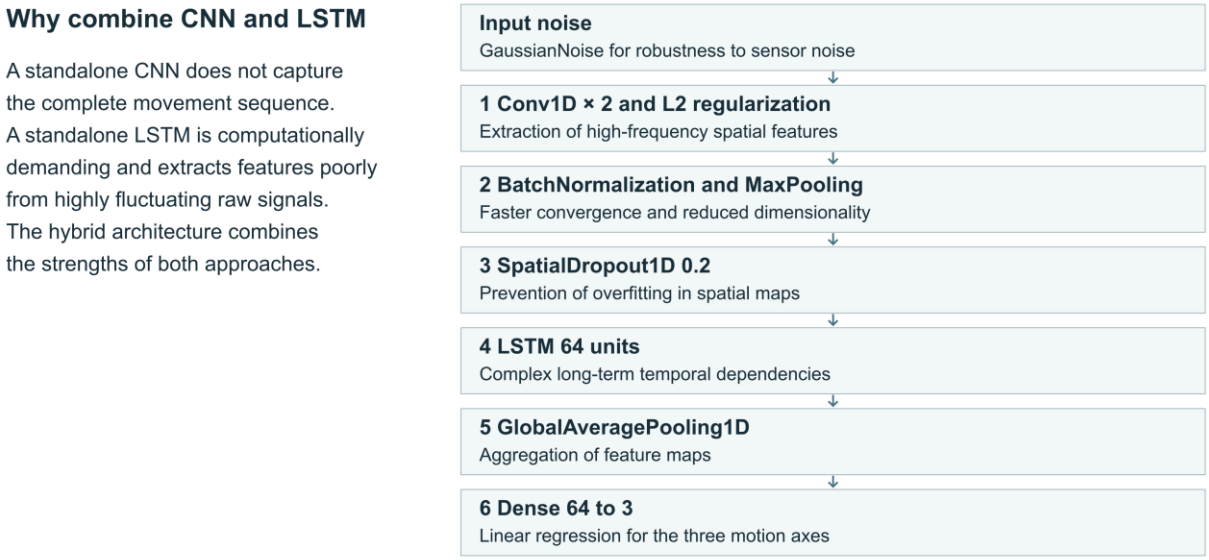



Figure 3.9 Detailed architecture of the final hybrid CNN-LSTM neural network

Taken together, the four phases represent a scientific development process. The project began with an initial architecture using discrete labels, demonstrated and confronted the difficulties of constructing a general model for new subjects, and ultimately returned to an optimized intra-subject approach. By incorporating biomechanical principles and calibration, it produced a practical and accurate model for continuous and simultaneous recognition of three hand motions.

# Chapter 4 Results and Evaluation

## 4.1 Quantitative Results of the Final Model

The final model converged to its optimal state at epoch 35 using learning rate reduction strategies such as ReduceLROnPlateau. Table 4.1 presents the evaluation indicators, including RMSE and the Pearson correlation coefficient, for all three motion axes on the test data.

Table 4.1 Quantitative evaluation results of the final model on the test data

| Motion axis | RMSE | Pearson correlation (CC) |
|---|---|---|
| Roll (rotation) | 0.1306 | 0.8707 |
| Pitch (flexion) | 0.0753 | 0.9634 |
| Yaw (deviation) | 0.1244 | 0.8949 |

As shown in Table 4.1, the highest accuracy was obtained for the Pitch axis, with a Pearson correlation coefficient of 0.9634, indicating that more than 96 percent of this axis's angular behavior was correctly predicted by the model. The Roll and Yaw axes also yielded acceptable results, with correlation coefficients of 0.8707 and 0.8949, respectively.

## 4.2 Visual Analysis of the Results

Visual inspection of the final model's output on the test data shows that the prediction curves for all three axes agree closely with the actual angles, without the substantial oscillations and high-frequency noise observed in Phases 2 and 3. Label smoothing enabled the network to capture the natural slope of muscular movement. Gain calibration allowed the predictions to settle stably at the maximum angles, without oscillation, within the range from −1 to +1. The limited residual noise at some points is likely attributable to hand tremor during dataset acquisition rather than a weakness of the model.

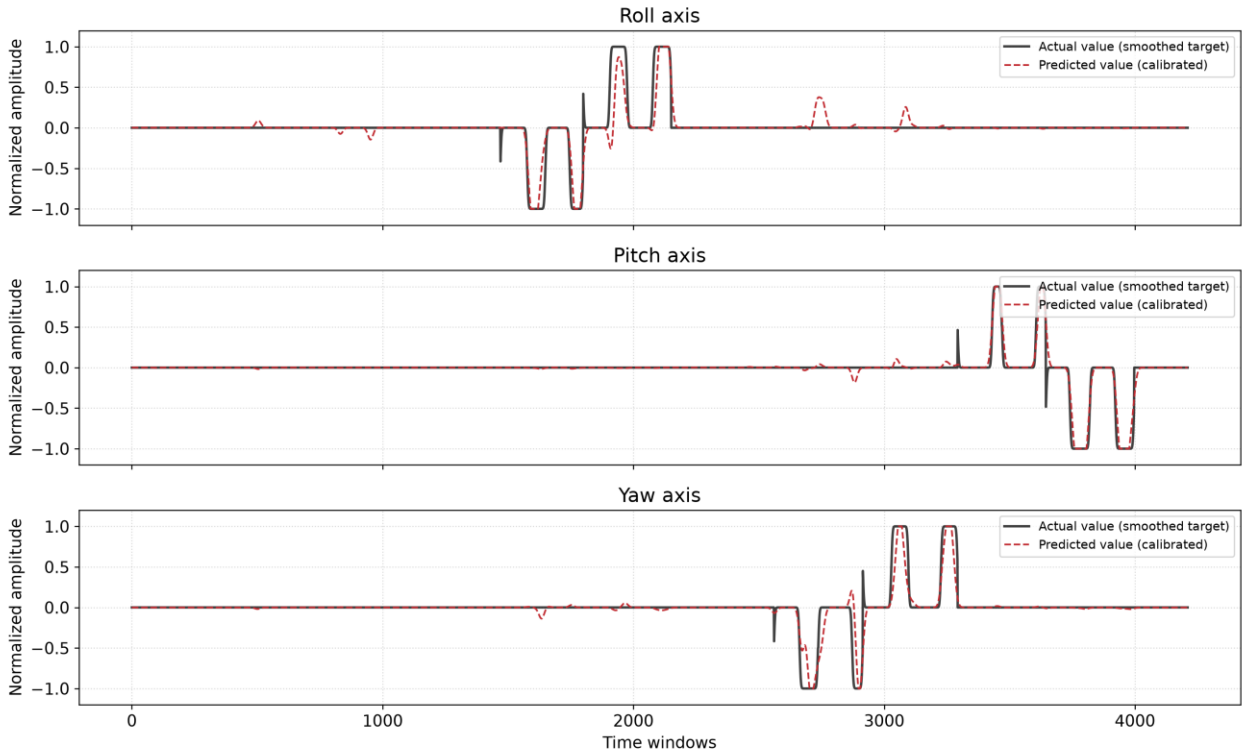



Figure 4.1 Agreement between actual and predicted angles of the final model on the test data

## 4.3 Discussion

The results of this project support two important scientific findings. First, the hybrid CNN-LSTM architecture with dilated convolutions is effective for jointly extracting spatial features and long-term temporal dependencies from sEMG signals. Second, covariate shift in biological signals across individuals is a real phenomenon that substantially limits general inter-subject training without advanced transfer learning techniques. Consequently, a subject-specific intra-subject training approach is the more reasonable choice for achieving the highest possible accuracy with the current technology.

# Chapter 5 Conclusions

## 5.1 Overview of the Findings

This project designed and developed a deep learning model for continuously and simultaneously estimating three wrist motion axes, namely Roll, Pitch, and Yaw, from sEMG signals. Development proceeded through four progressive phases, in which the network evolved from a simple intra-subject model to a general inter-subject model and finally to an advanced intra-subject model with continuous outputs. The principal achievements include addressing data leakage in Phase 1, experimentally demonstrating covariate shift in Phase 3, and introducing two key innovations in Phase 4: Gaussian label smoothing and gain calibration. On the test data, the final model achieved a Pearson correlation coefficient above 0.87 on all three motion axes, which the study considers a reliable value for practical implementation.




Figure 5.1 Overview of the project's development through four phases

## 5.2 Limitations

- Because the final model uses intra-subject training, it cannot be applied directly to new individuals without calibration or readjustment.
- Covariate shift in sEMG signals across individuals remains unresolved, and the methods used in this project did not enable the construction of a fully general model.
- The final evaluation was based on laboratory data from the NinaPro DB2 database. Model performance in real-world and noisier operating conditions has not yet been examined.

## 5.3 Recommendations for Future Research

- Investigate more advanced transfer learning and domain adaptation methods to reduce the effects of covariate shift across individuals.
- Implement the model on embedded hardware with limited computational resources for use in portable intelligent prostheses.
- Collect a dedicated dataset from a larger number of users under more varied acquisition conditions to evaluate model generalization more thoroughly.
- Investigate Transformer-based architectures for modeling long-term temporal dependencies in sEMG signals.

# Chapter 6 Verification of the Supplied Implementation

Chapters 1–5 present the complete original study. The supplementary analysis in Chapters 6–8 extends that study through inspection of the archived implementation, saved model configuration, execution logs, supplied reference documents, and exported target and prediction arrays. These chapters distinguish additional analyses of existing evidence from proposed experiments. No additional model training on raw NinaPro data or clinical deployment experiment is claimed.

## 6.1 Evidence and Reproducibility Scope

The submitted archive contains one executable research notebook for Phase 4, a Python program for three-dimensional visualization, one trained Keras model, two NumPy sample arrays, a demonstration video, twelve report figures, a report, a presentation, seven research papers, and an offline copy of the DB2 database description. It does not contain the raw MATLAB recordings used for training. Consequently, the model architecture, numerical postprocessing, saved prediction behavior, document coverage, and software organization can be examined directly, whereas raw-data training and the earlier phases cannot be independently rerun from the archive alone.

A reproducibility claim must specify which evidence supports it. The existence of a trained model establishes that model parameters were saved, but it does not establish the identity of every training observation, the reproducibility of the optimization trajectory, or the independence of the evaluation partition. Similarly, a recorded animation demonstrates a visualization of saved trajectories, but does not measure the latency of a streaming acquisition-and-inference system. The additional analysis therefore retains the original reported results while explicitly limiting new conclusions to the available artifacts.

Table 6.1 Evidence available for verification

| Artifact | Available evidence | Verification limit |
|---|---|---|
| Phase 4 notebook | Source operations and stored execution outputs | Raw recordings and the original environment are absent |
| Trained Keras model | Layer configuration, weights package, save metadata | No independent training rerun |
| Saved sample arrays | 4,211 paired observations with three columns | No original timestamps, repetitions, or subject labels in these arrays |
| Report and presentation | Original explanations, results, references, and figures | Several methodological statements require qualification |
| Earlier development phases | Narrative descriptions and output figures | Executable Phase 1–3 implementations were not submitted |
| Demonstration video | Recorded comparison of two animated hand representations | No physical-angle calibration or deployment timing measurements |

## 6.2 Target Construction and the Meaning of the Outputs

The supplied Phase 4 implementation reads the variables emg, restimulus, and rerepetition. Movement identifiers are converted into a three-column target array. Codes 9 and 10 assign negative and positive Roll values, codes 15 and 16 assign positive and negative Pitch values, and codes 13 and 14 assign negative and positive Yaw values. Rest and other unmapped codes receive zero values on all three axes. A Gaussian filter is then applied independently to each target column with sigma 400. The resulting trajectories are continuous numerical surrogates for movement activation rather than direct samples of measured wrist angles.

Table 6.2 Movement code mapping used by the implementation

| Movement code | Roll | Pitch | Yaw |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 9 | −1 | 0 | 0 |
| 10 | +1 | 0 | 0 |
| 15 | 0 | +1 | 0 |
| 16 | 0 | −1 | 0 |
| 13 | 0 | 0 | −1 |
| 14 | 0 | 0 | +1 |

This distinction affects the interpretation of every numerical result. A low regression error indicates agreement with the label-derived target trajectory. It does not independently establish accuracy in degrees, agreement with a physical goniometer, or correspondence to continuously measured Cyberglove joint angles. Smoothing supplies a transition shape chosen by the preprocessing procedure. It cannot reconstruct unobserved intermediate physical motion from a discrete movement identifier. The method remains useful for investigating proportional control signals, provided the outputs are described as normalized motion estimates and the target-generation assumptions are made explicit.

At a sampling frequency of 2,000 Hz, sigma 400 corresponds to a nominal smoothing scale of 0.2 seconds before windowing. This is a description of the filter parameter, not a measured response delay. A centered Gaussian filter uses samples on both sides of a time point. The current target-generation procedure is therefore suitable for offline analysis, while a causal control system would require a separately defined transition model or directly measured kinematic supervision.

## 6.3 Preprocessing and Windowing Audit

The source first selects the movement codes of interest and rest, thereby constructing a reduced time series. It rectifies this signal, applies a fourth-order Butterworth low-pass filter with a cutoff of 5 Hz, and takes the absolute value of the filtered result. StandardScaler is fitted only to samples whose repetition identifiers belong to the training set. The fitted transform is then applied to the complete selected sequence. These exact

operations are retained in the organized implementation so that structural cleanup does not silently redefine the original experiment.

Selection before filtering can concatenate samples that were separated by discarded movements in the original recording. Filtering across a newly created adjacency can introduce transitions that did not exist in the original physical timeline. Moreover, a zero-phase filter applied to the entire selected sequence can use information across training and evaluation boundaries. Fitting the scaler only on training samples addresses leakage through normalization, but does not by itself isolate filtering, target smoothing, or the temporal content of overlapping windows.

The time window contains 600 samples, or 300 milliseconds at the stated sampling frequency. The Phase 4 step is 100 samples, corresponding nominally to 50 milliseconds between window starts. Adjacent windows overlap by 500 samples, or approximately 83.3 percent of their length. Such overlap increases the number of training examples but also creates strong dependence between neighboring observations. The implementation uses the repetition identifier at the final sample to assign an entire window to training or evaluation. It does not require all 600 samples to have the same repetition or partition identifier.

The additional recording-audit utility detects removed time gaps and counts windows containing multiple repetition identifiers or crossing discarded intervals. These are diagnostic counts, not automatic proof of harmful leakage in every counted window. A mixed repetition may occur within the same partition, whereas a cross-partition window has stronger implications for evaluation independence. Because the raw recording is absent, the utility has been tested on a synthetic recording but its counts for the actual NinaPro experiment remain undetermined.

## 6.4 Verified Final Architecture and Parameter Counts

The saved model configuration and the archived model summary agree on a compact CNN-LSTM architecture containing 45,923 parameters in total. Of these, 45,731 are trainable and 192 are nontrainable BatchNormalization statistics. The two convolutional layers use dilation rate 1. The recurrent layer is a unidirectional LSTM with 64 units. Consequently, the supplied final model should be distinguished from the dilated and bidirectional architectures described in the intermediate development phases.

Table 6.3 Layer dimensions and parameter counts in the final model

| Layer | Output dimensions excluding batch | Parameters |
|---|---|---|
| Input and GaussianNoise | 600 × 12 | 0 |
| Conv1D with 32 filters and kernel 5 | 596 × 32 | 1,952 |
| BatchNormalization | 596 × 32 | 128 |
| MaxPooling1D | 298 × 32 | 0 |
| SpatialDropout1D 0.2 | 298 × 32 | 0 |
| Conv1D with 64 filters and kernel 3 | 296 × 64 | 6,208 |
| BatchNormalization | 296 × 64 | 256 |

| Layer | Output dimensions excluding batch | Parameters |
|---|---|---|
| MaxPooling1D | 148 × 64 | 0 |
| SpatialDropout1D 0.2 | 148 × 64 | 0 |
| LSTM with 64 units | 148 × 64 | 33,024 |
| Dropout 0.3 | 148 × 64 | 0 |
| GlobalAveragePooling1D | 64 | 0 |
| Dense with 64 units | 64 | 4,160 |
| Dropout 0.3 | 64 | 0 |
| Dense with three linear outputs | 3 | 195 |
| Total | Three normalized outputs | 45,923 |

The first convolution has 32 filters, each containing five time positions across twelve input channels and one bias, giving 1,952 parameters. The second convolution receives 32 feature channels and contains 64 filters of length three, giving 6,208 parameters. For an LSTM with 64 input features and 64 hidden units, the four gates together account for 33,024 parameters. The recurrent layer therefore represents approximately 71.9 percent of the model's total parameter count. This explains why the recurrent component is the principal storage contribution even though convolution reduces the sequence from 600 to 148 positions before recurrent processing.

The one-dimensional convolution kernels move along the temporal dimension and jointly use the input feature channels. Describing them solely as spatial feature extractors is an architectural shorthand; they also extract local temporal patterns. The LSTM models dependencies among the resulting temporal feature vectors, while global average pooling aggregates the recurrent sequence before regression. Pooling and convolution reduce the number of recurrent positions, but parameter counts alone do not establish execution time or energy consumption. Those quantities require profiling on a specified device.

The model metadata records Keras version 3.14.0 and a save time of 2026-06-04 at 22:24:54. The stored optimizer learning rate is approximately 0.0000625, consistent with reductions from the executed initial rate of 0.00025. In the notebook, an earlier compile at 0.0005 is replaced by a second compile before training; only the latter is effective for the recorded training call. The organized code removes the redundant compile while retaining the effective setting.

## 6.5 Training and Evaluation Provenance

The archived execution log states that the best validation-loss checkpoint occurred at epoch 35 and that early stopping occurred at epoch 50. This supports the source report's identification of epoch 35 as the selected checkpoint. The validation loss includes the contribution of the model's regularization terms and should not be interpreted directly as the unregularized RMSE of the three outputs. In addition, validation data are reused

by the callbacks and by the final reported evaluation, so the evaluation partition participates in model selection.

An inconsistency is visible in the stored outputs. The preprocessing cell reports 8,689 training windows and 4,211 evaluation windows, while the training trace reports 198 batches per epoch. With the current source batch size of 64, 8,689 observations would require 136 batches. This discrepancy may reflect retained outputs from a different execution or an earlier dataset or batch configuration. The saved outputs are preserved as historical evidence, but they cannot certify that all notebook cells and the checkpoint came from one fully consistent execution of the current source.

The organized workflow retains the original validation behavior for reproduction and documents it explicitly. It saves new training checkpoints to a different path, exports both pre-calibration and calibrated metrics, and keeps the supplied model and sample arrays unchanged. A scientifically stronger future experiment should select checkpoints and any calibration parameters using an independent validation partition and reserve an untouched test partition for the final evaluation.

## 6.6 Comparison with the Supplied Literature

The reference files support the main methodological themes of the project but use different datasets and evaluation tasks. Bao et al. [3] combine convolutional feature extraction with LSTM sequence regression and evaluate wrist kinematics using data from six healthy participants. Quivira et al. [4] use recurrent networks with sEMG and hand-pose measurements from Myo and Leap Motion sensors across seven users. Du et al. [5] evaluate a TCN-LSTM architecture on NinaPro DB1 using the first eight subjects and selected wrist-related movements. These studies support temporal modeling, but their reported scores are not directly comparable to the present DB2 surrogate-label experiment.

Table 6.4 Methodological relevance of the supplied references

| Reference | Principal evidence in the supplied paper | Implication for this project |
|---|---|---|
| [1] Atzori et al. | Benchmark acquisition and myoelectric movement classification | Provides database context; the archived copy is a manuscript version |
| [2] Gijsberts et al. | Movement error rate and prediction-delay analysis | Control evaluation should consider error type and timing |
| [3] Bao et al. | CNN-LSTM wrist kinematics estimation with six participants | Supports combining local feature extraction with recurrent sequence modeling |
| [4] Quivira et al. | Recurrent estimation of hand poses using Myo and Leap Motion | Illustrates supervision with independently measured kinematics |
| [5] Du et al. | TCN-LSTM evaluation on NinaPro DB1 with eight subjects | Supports investigation of temporal convolutions; its dataset differs from DB2 |
| [6] Ameri et al. | Regression CNN evaluated through an online Fitts' law task | Shows the importance of evaluating functional control performance |
| [7] Atzori et al. | Convolutional classification on intact and amputee recordings | Supports feature-learning research while using a classification task |

A convolutional model's limitations should therefore be stated relative to its particular architecture and input representation, rather than as an absolute inability of all CNNs to model time. Likewise, the observed failure of the project's inter-subject models demonstrates a limitation of those experiments; it does not establish that subject-independent sEMG estimation is impossible in principle. Domain adaptation, personalized calibration, richer supervision, and alternative validation designs remain hypotheses requiring evaluation.

# Chapter 7 Extended Evaluation of the Saved Predictions

## 7.1 Data Integrity and Evaluation Definitions

The supplementary numerical analysis uses the unchanged files y_true_sample.npy and y_pred_sample.npy. Both contain 4,211 observations with columns ordered as Roll, Pitch, and Yaw. The target array uses 64-bit floating-point values and the prediction array uses 32-bit floating-point values. All entries are finite. The predictions lie within the clipping range [−1, +1]. The target extremes differ from that range only by negligible floating-point rounding. No observations are removed from the overall metrics.

For each axis, RMSE is calculated as the square root of the mean squared prediction error. MAE is the mean absolute prediction error, and signed mean error measures systematic bias with the sign convention prediction minus target. The 95th percentile of absolute error describes the upper tail of typical errors, while maximum absolute error identifies the largest discrepancy among the saved observations. Pearson correlation measures linear association. The coefficient of determination is calculated as one minus the ratio of residual squared error to target variation around the target mean. It can be negative when prediction is worse than the mean-target reference and is undefined when the target is constant.

All error values are expressed in normalized target-amplitude units. They are not degrees, percentages of correctly classified movements, or clinical performance scores. Because the saved observations derive from overlapping windows and their original timestamps are unavailable, confidence intervals based on independent-observation assumptions would be inappropriate. No such confidence intervals or statistical significance claims are introduced in the supplementary evaluation.

Table 7.1 Extended metrics for all saved calibrated predictions

| Axis | RMSE | MAE | Pearson CC | R squared |
|---|---|---|---|---|
| Roll | 0.1181 | 0.0315 | 0.8766 | 0.7659 |
| Pitch | 0.0554 | 0.0142 | 0.9697 | 0.9392 |
| Yaw | 0.1059 | 0.0269 | 0.8981 | 0.7967 |

Pitch has the lowest overall RMSE and MAE and the highest correlation and coefficient of determination. Roll and Yaw show larger aggregate errors. The joint mean absolute error across all observations and all three columns is 0.0242. This average is useful for describing the saved output array, but it must be interpreted together with activity-conditioned and tail-error diagnostics because the arrays contain many observations whose individual axis targets are near zero.

## 7.2 Relationship to the Originally Reported Results

Table 7.2 Original pre-calibration results and saved calibrated results

| Axis | Original RMSE | Saved RMSE | Original CC | Saved CC |
|---|---|---|---|---|
| Roll | 0.1306 | 0.1181 | 0.8707 | 0.8766 |
| Pitch | 0.0753 | 0.0554 | 0.9634 | 0.9697 |
| Yaw | 0.1244 | 0.1059 | 0.8949 | 0.8981 |

The original notebook calculates its printed metrics after smoothing the predicted outputs, before applying gain calibration. It then multiplies those predictions by 1.35, clips them to [−1, +1], and saves the calibrated array. Recomputing metrics from the saved array therefore evaluates a different postprocessing stage. The saved calibrated outputs have lower RMSE on all three axes than the values printed in the archived notebook. This is a descriptive comparison of the available evidence rather than a newly controlled calibration experiment.

Multiplication by a positive constant alone preserves Pearson correlation because it scales covariance and prediction standard deviation by the same factor. Clipping is nonlinear and can change correlation. The small changes in correlation between the original and saved results are therefore consistent with a pipeline containing clipping. The exact pre-calibration trajectory cannot be recovered by dividing the saved values by 1.35, since clipping discards information about how far the original amplified prediction exceeded the permitted limits.

## 7.3 Residual Behavior and Large Errors

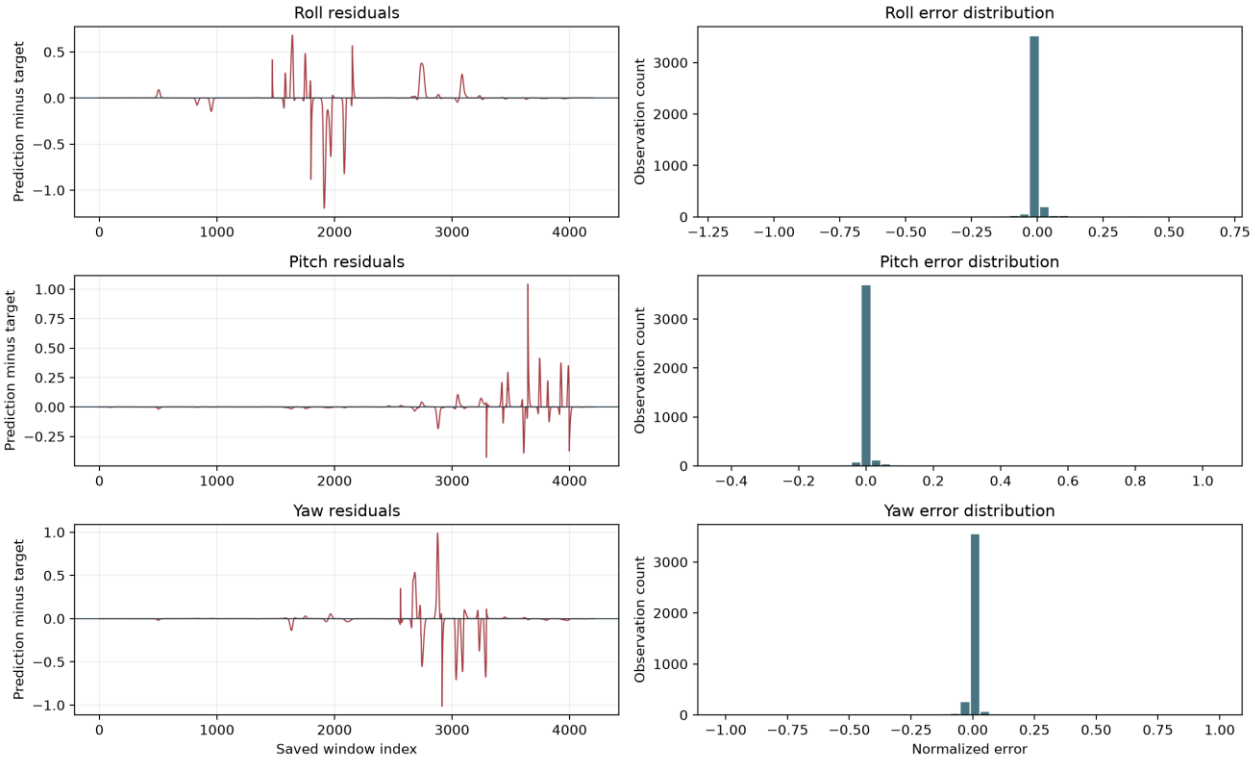



Figure 7.1 Residual trajectories and error distributions for the saved calibrated predictions

Table 7.3 Bias and absolute-error tails

| Axis | Signed mean error | 95th percentile absolute error | Maximum absolute error |
|---|---|---|---|
| Roll | -0.0017 | 0.2010 | 1.1960 |
| Pitch | 0.0040 | 0.0737 | 1.0428 |
| Yaw | -0.0062 | 0.1306 | 1.0137 |

The signed mean errors are small relative to the full normalized range, but small average bias does not imply small error at every movement boundary. Positive and negative discrepancies can cancel when averaged. The maximum absolute error exceeds 1.0 on each axis, despite the high correlations reported for the complete trajectories. This is possible because the full distance between opposite normalized extremes is 2.0. Large isolated discrepancies can coexist with good overall agreement when most observations have low error.

The residual figures therefore complement the full-trajectory comparison in Chapter 4. They reveal when the prediction departs from the target and show whether the distribution concentrates near zero or contains a substantial tail. The archived arrays do not provide the raw EMG, original timestamps, repetition identifiers, or acquisition events needed to determine the physical cause of every error peak. In particular, residual errors cannot be attributed conclusively to hand tremor, sensor noise, a model failure, or discontinuities introduced by selecting and concatenating evaluation windows.

## 7.4 Prediction and Target Agreement

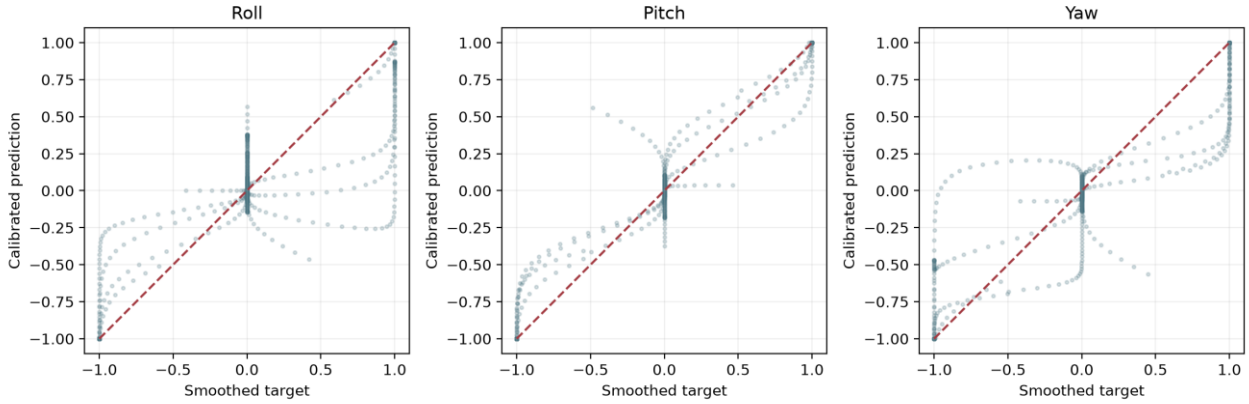



Figure 7.2 Scatter comparison between smoothed targets and saved calibrated predictions

Each scatter diagram compares the target and prediction at the same saved observation index. The diagonal represents perfect numerical agreement. Points near the diagonal indicate matching amplitudes, while departures show underestimation, overestimation, or mismatched signs. Dense clusters near zero reflect the frequency of near-zero targets rather than a uniformly sampled distribution of physical joint angles. Horizontal saturation bands at the prediction limits are a consequence of clipping and should not be interpreted as independent evidence of accurate endpoint control.

A trajectory correlation primarily reflects association over the complete sample sequence. It is insensitive to several amplitude and offset changes, and it does not capture all forms of local control failure. The scatter plots, residual plots, RMSE, MAE, and error-tail statistics evaluate complementary aspects of the same predictions. Their joint interpretation is more informative than describing a single correlation coefficient as percentage accuracy.

## 7.5 Activity Conditioned Evaluation and the Zero Baseline

To examine the effect of sparse movement activation, observations are grouped independently for each axis. An axis is designated active when the absolute value of its smoothed target exceeds 0.05; all other observations are designated near rest for that axis. This threshold is an explicit diagnostic convention, not a physiological threshold validated against muscle activity. A near-rest Roll target can occur while Pitch or Yaw is active, so these counts describe individual target columns rather than complete hand rest.

Table 7.4 Activity counts and conditional errors

| Axis | Active samples | Near-rest samples | Active RMSE | Near-rest RMSE |
|---|---|---|---|---|
| Roll | 320 | 3891 | 0.3939 | 0.0483 |
| Pitch | 286 | 3925 | 0.1893 | 0.0262 |
| Yaw | 303 | 3908 | 0.3740 | 0.0351 |

Only 320 Roll observations, 286 Pitch observations, and 303 Yaw observations exceed the selected threshold. These represent approximately 7.6, 6.8, and 7.2 percent of the respective target columns. The active-state RMSE is substantially larger than the near-rest RMSE on every axis. This finding demonstrates why low aggregate error must not be interpreted as equally low error throughout the active movements. The diagnostic should be repeated with alternative thresholds and with movement and repetition labels once the raw recording is available.

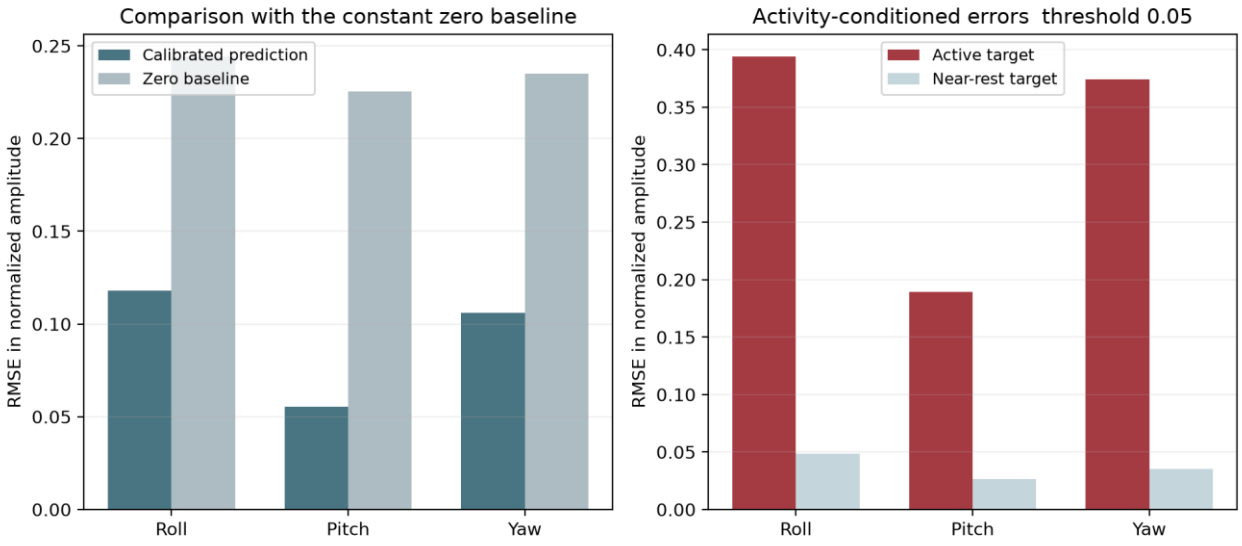



Figure 7.3 Zero-baseline comparison and activity-conditioned RMSE

Table 7.5 Constant zero baseline and prediction saturation

| Axis | Calibrated RMSE | Zero baseline RMSE | Predictions at clipping limits |
|---|---|---|---|
| Roll | 0.1181 | 0.2441 | 91 |
| Pitch | 0.0554 | 0.2254 | 118 |
| Yaw | 0.1059 | 0.2348 | 58 |

A constant zero predictor is included because zero is the assigned target for an inactive axis and is frequent in the saved arrays. Its overall RMSE is 0.2441 for Roll, 0.2254 for Pitch, and 0.2348 for Yaw. The saved model predictions improve on this baseline on every axis. The comparison confirms that the low reported errors are not achieved merely by predicting zero throughout the sequence, while the activity-conditioned results show that substantial movement-specific error remains.

There are 91 Roll predictions, 118 Pitch predictions, and 58 Yaw predictions at the clipping limits within an absolute tolerance of 0.0000001. Saturation is part of the designed postprocessing and does not by itself constitute an error. Its control implications depend on whether the actual target is also near an endpoint, whether saturation persists for an appropriate duration, and whether the output recovers smoothly during transitions. Those questions require a temporal control evaluation rather than an aggregate count alone.

# Chapter 8 Reproducible Software and Further Research Design

## 8.1 Organization of the Research Implementation

The organized implementation separates experiment configuration, preprocessing, model construction, training, evaluation, diagnostics, and visualization. Paths are derived from the project location rather than a fixed drive-specific directory. The training function writes a new checkpoint and rejects the path of the supplied trained model. Saved-sample evaluation and model-package inspection can be performed without training, while raw-recording preparation checks the expected variables and the twelve-channel input shape before processing.

Table 8.1 Responsibilities of the organized Python modules

| Module | Responsibility |
|---|---|
| config.py | Project paths, subject, window length, step, filter, smoothing, and training parameters |
| preprocessing.py | MATLAB loading, movement selection, filtering, scaling, targets, and window assignment |
| model.py | Final CNN-LSTM construction and training callbacks |
| evaluation.py | RMSE, correlation, smoothing, gain calibration, clipping, and trajectory plots |
| diagnostics.py | Expanded error statistics, activity analysis, model-package inspection, and recording audit |
| visualization.py | Three-dimensional replay of saved normalized trajectories |
| __main__.py | Command-line interfaces for the reproducible workflows |

The documented notebook exposes the same reusable functions rather than duplicating the training implementation in separate cells. It first permits inspection of the supplied saved samples, then presents the raw-data workflow. New outputs are written to their own run folder. The original textual outputs are retained separately as archived evidence, while the reorganized notebook clears stale execution results so that its displayed code is not falsely presented as a newly completed training run.

## 8.2 Verification of Numerical Equivalence

Thirteen regression and diagnostic tests were executed successfully. They verify the Butterworth transformation against the original mathematical operation, the signs and column ordering of the movement mapping, Gaussian smoothing, the exclusive window bound, final-sample target alignment, gain multiplication, clipping, saved-array integrity, and rejection of mismatched or nonfinite inputs. Additional tests examine known metric cases, constant targets, the saved model configuration, checkpoint protection, missing-data errors, and recording-audit detection of discontinuities.

A full preprocessing comparison uses a synthetic twelve-channel MATLAB recording containing selected movements, an unsupported movement code, and all six repetition identifiers. An independent transcription of the source operations produces training and evaluation arrays that match the organized implementation exactly. This test provides evidence that the refactoring preserves the preprocessing calculations and split behavior. It does not certify the quality of the source validation methodology or reproduce a trained model's performance on the absent raw NinaPro recording.

The Word and PDF reports were checked for complete retention of the source text, original table rows, figures, and reference entries. The presentation retains all fifteen original slide topics with translated editable text and a native result table. The model, two NumPy arrays, and demonstration video match their source SHA-256 checksums. These checks address completeness and artifact integrity; they are distinct from independent scientific validation of the experimental claims.

## 8.3 Independent Training Validation and Test Partitions

A stronger intra-subject experiment should reserve separate repetitions for training, validation, and final testing before fitting any data-dependent transform. One possible design uses repetitions 1, 3, and 6 for training, repetition 4 for validation, and repetitions 2 and 5 for testing. This is a proposed design, not a completed replacement experiment. Its suitability depends on checking the available rest intervals, movement coverage, and the structure of rerepetition in the original recording.

The scaler should be fitted on training samples only. Filtering and smoothing should be applied within appropriately isolated continuous segments, or implemented causally for streaming evaluation. Windows should be required to remain within one permitted segment and partition. Checkpoint selection, learning-rate decisions, gain calibration, and hyperparameter selection should use validation data only. Test labels should remain unavailable to these decisions. The final test report should describe both overall performance and results grouped by movement, repetition, and activity level.

For inter-subject evaluation, the outer test subject must remain independent of both training and model selection. If adaptation uses examples from that subject, the adapted examples must be separated from the examples used for evaluation. A nested design can distinguish an unadapted model, a model calibrated with a fixed small support set, and a subject-specific model trained with a larger data budget. The amount and type of personal calibration data should be reported because generalization without adaptation and performance after adaptation answer different scientific questions.

## 8.4 Measured Kinematic Supervision and Physical Calibration

To extend the study from proportional movement surrogates to measured wrist-angle regression, a future dataset should supply synchronized physical kinematic targets with an explicit mapping between sensors and anatomical axes. The study should document the reference pose, angle units, sign conventions, valid ranges, missing samples, sensor calibration, and synchronization procedure. Device channels that measure finger motion should not be treated automatically as direct measurements of every wrist or forearm degree of freedom.

Target normalization should be based on a documented physical range or calibration protocol rather than an assumed universal maximum. Prediction error can then be reported in degrees alongside normalized metrics. A separate evaluation can determine whether smoothing improves noise robustness at the expense of

transition delay. This distinction is particularly important when the target filter imposes a trajectory shape that the network can reproduce even if the timing of the actual joint motion differs.

## 8.5 Ablation Experiments and Baseline Comparisons

Table 8.2 Proposed experiments and the questions they address

| Experiment | Controlled comparison | Evidence to report |
|---|---|---|
| Target smoothing | Discrete labels versus several Gaussian smoothing scales | Overall and transition-specific error with clearly defined targets |
| Prediction smoothing | No smoothing versus causal and offline filters | Error, response delay, and peak attenuation |
| Gain calibration | No gain versus validation-selected gain with and without clipping | Both pre-calibration and final test metrics |
| Architecture | CNN, LSTM, CNN-LSTM, and a specified TCN-LSTM | Identical partitions, parameter counts, inference cost, and error |
| Personal adaptation | No adaptation versus fixed calibration budgets | Held-out subject performance on examples unused for adaptation |
| Signal robustness | Controlled noise, electrode displacement, and session variation | Performance degradation and recovery under specified perturbations |

Each experiment should change one identified factor while holding the relevant partitioning and evaluation procedure constant. The zero predictor provides a minimal reference for the sparse target arrays, but stronger baselines should include suitable linear or conventional regression methods and simpler neural architectures. Comparison should account for both predictive performance and computational cost. A larger model or higher correlation is not automatically preferable when latency, memory, personal calibration requirements, and control stability affect the application.

Repeated training with different random seeds can characterize optimization variability once the data and environment are available. Reporting only the best seed would conceal that variability. For subject-level conclusions, variability across subjects is more relevant than treating overlapping windows as independent experimental units. Any confidence interval or hypothesis test should therefore use a resampling or hierarchical design that respects the subject, repetition, and temporal dependence of the observations.

## 8.6 Causal Streaming and Deployment Evaluation

The present offline pipeline includes forward-backward filtering and centered smoothing, so its complete preprocessing cannot be executed without access to future samples. A deployable streaming version must replace those stages with causal filters or define a fixed and reported look-ahead delay. The 300-millisecond input window and nominal 50-millisecond stride describe the source configuration but are not sufficient to determine end-to-end control latency. Acquisition, filtering, window completion, inference, communication, and actuation each contribute to the measured delay.

The three-dimensional demonstration replays the saved sequence at forty frames per second and maps normalized outputs to an illustrative ±90-degree range. That playback rate corresponds to twenty-five milliseconds per displayed frame, while the nominal source window stride is fifty milliseconds. In addition, the exported evaluation sequence does not preserve the complete original timestamp series. The animation should therefore be used as a qualitative comparison, not as proof of physical-angle accuracy or real-time timing.

Deployment experiments should identify the processor, runtime, numerical precision, memory footprint, measured inference time, and worst-case end-to-end delay. They should assess false motion during rest, unintended reversals, saturation, transition tracking, recovery after noise, and the effect of recalibration. An interactive target-reaching task, analogous in purpose to the functional evaluations discussed by Ameri et al. [6], would provide evidence beyond offline trajectory agreement. No such deployment measurement is available in the submitted archive.

## 8.7 Conditions for Further Scientific Conclusions

The supplied evidence supports a reproducible offline analysis of normalized movement-label-derived trajectories for the saved subject-specific outputs. The supplementary evaluation confirms improvement over the constant zero baseline and quantifies residual error, sparse activation, and clipping. It also identifies limitations that must be resolved before stronger conclusions about measured wrist angles, unseen subjects, causal streaming, or prosthesis control can be drawn. Future work should extend the evidence through independently supervised kinematics, isolated validation and test partitions, and functional deployment experiments rather than relying solely on longer narratives or additional model complexity.

# References

[1] M. Atzori, A. Gijsberts, I. Kuzborskij, S. Elsig, A.-G. M. Hager, O. Deriaz, C. Castellini, H. Müller, and B. Caputo, "Characterization of a Benchmark Database for Myoelectric Movement Classification," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 23, no. 1, pp. 73–83, 2015.

[2] A. Gijsberts, M. Atzori, C. Castellini, H. Müller, and B. Caputo, "Movement Error Rate for Evaluation of Machine Learning Methods for sEMG-Based Hand Movement Classification," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 22, no. 4, pp. 735–744, 2014.

[3] T. Bao, S. A. R. Zaidi, S. Xie, P. Yang, and Z. Zhang, "A CNN-LSTM Hybrid Framework for Wrist Kinematics Estimation Using Surface Electromyography," arXiv preprint arXiv:1912.00799, 2019.

[4] F. Quivira, T. Koike-Akino, Y. Wang, and D. Erdogmus, "Translating sEMG Signals to Continuous Hand Poses Using Recurrent Neural Networks," in Proc. IEEE EMBS International Conference on Biomedical & Health Informatics (BHI), 2018, pp. 166–169.

[5] J. Du, Z. Liu, W. Dong, W. Zhang, and Z. Miao, "A Novel TCN-LSTM Hybrid Model for sEMG-Based Continuous Estimation of Wrist Joint Angles," Sensors, vol. 24, no. 17, p. 5631, 2024.

[6] A. Ameri, M. A. Akhaee, E. Scheme, and K. Englehart, "Regression Convolutional Neural Network for Improved Simultaneous EMG Control," Journal of Neural Engineering, vol. 16, no. 3, 036015, 2019.

[7] M. Atzori, M. Cognolato, and H. Müller, "Deep Learning with Convolutional Neural Networks Applied to Electromyography Data: A Resource for the Classification of Movements for Prosthetic Hands," Frontiers in Neurorobotics, vol. 10, p. 9, 2016.